04 · Proof & traction — what is built and proven

04 · Proof & traction — what is built and proven

> *Confidential DD dossier. Everything marked **PROVEN** is verifiable by reading the `arc-lean` source and running `zig build test` / `bash scripts/build.sh`. Founder-reported items awaiting independent replication are marked as such.*

The build gate (the honesty mechanism)

The whole substrate ships behind one gate: `scripts/build.sh` runs the unit test suite, **53 doctrine-parity claims** (mechanical checks that the code matches its own stated doctrine), a formatting pass, and a **wallet-signed self-attestation** of the binary. A green gate means every module's in-source tests pass *and* the code has not drifted from its documented invariants. This dossier's **PROVEN** tags are all downstream of that gate. It is currently green: *53/53 claims, signed.*

The falsifiable atom — energy-ranked resolution beats scan-order

The single most important proof, because the entire thesis rests on it: **does the collective's energy prior actually route thinking better than arbitrary order?**

- **Result: yes, measured.** On a prove-or-kill benchmark (producer/consumer contract graph), energy-ranked resolution settles the graph in **2 passes vs 3** for scan-order — fewer cascades, less wasted work — **16/16** runs.
- **Mutation-verified:** reversing the energy sort restores 3 passes. So the *ranking*, not luck or an unrelated change, is doing the work.
- **Non-breaking:** it is reorder-only; every candidate still fires (fairness floor holds). It is now **always on** — the default resolution path.

This is "the collective routes its own thinking," proven at the atom before building the cathedral. **PROVEN** (`main.zig` router; `examples/arc-agi3/ bench-energy-router.sh`).

The product one-shot — Wine/Overwatch on macOS

The harness (with Opus as the reasoner inside it) **rebuilt a Wine compatibility layer to launch and run Overwatch on a Mac, in effectively one shot** — a competitive Windows title brought up on macOS from a standing start. This is the capability proof: most agents cannot do this, and you cannot get there by telling a general agent "make Overwatch run on my Mac." It worked because the harness forced first-principles, layer-by-layer decomposition and fault localization. **Founder-demonstrated** (a Loom/recording is the artifact to request for diligence).

ARC-AGI benchmark — reasoning without a large model

The founder tested an earlier version against **ARC-AGI-3** (a benchmark designed so that passing normally requires a large model) and **reached a high score without using any large model** on the order of Grok/Opus — the point being that the *harness*, not raw parameter count, carried the reasoning. **Founder-reported; awaiting independent replication** (the arc-lean tree is mid-rebuild toward that earlier capability — `examples/arc-agi3/`). Diligence should request the exact score, the task split, and a reproducible harness invocation.

The commons economics — proven locally, end to end

The cooperative-commons economics (`ECONOMICS-cooperative-commons.md`) are not a whitepaper sketch; the mechanism runs and is tested:

- **Reward, not slash.** Usage fees form a pool paid out to the maintainers whose routes were *used* and still *verify* — usage-weighted, verification-gated, stake-gated. Tested invariants: **conservation** (nothing burned), **forfeit -> value** (a route that fails re-verification forfeits its fees to the honest, who then earn more than their own usage), **stake-gated** (unstaked earns nothing). **PROVEN** (`reward.zig`).

- **Wash-trading is a donation, not a theft.** Sybil-dampened usage accounting (self-exclusion + per-payer cap): a maintainer wash-calling its own route earns **0** and the honest take the pool. **PROVEN** (`usage.zig`, `usage_ledger.zig`).
- **The whole loop runs from the real ledger.** `epoch.runEpoch` + `runEpochFromLedger` assemble stake (from chain), owner + attestation + usage (from ledger rows) and produce the payout set — a capstone test drives every gate at once from JSONL. **PROVEN.**

Real chain integration — reads live, writes are operator-gated

- **FDRY staking is real.** `voltr.zig` builds native Voltr vault instructions against the **real mainnet vault**; its PDA derivation **reproduces the actual on-chain vault authority** from seeds — a wrong byte fails the build. Cross- validated against Voltr's published CPI reference (program id, discriminators, account order, seeds all match). **PROVEN, without spending anything.**
- **Live reads work.** `solana_rpc` fetches live devnet blockhashes and reads a wallet's stFDRY stake from mainnet (`stake.zig`). **PROVEN.**
- **The submit path is cluster-valid.** `broadcast.zig` compiles + signs a legacy transaction; submitting it to devnet with a dummy blockhash returns `BlockhashNotFound` — i.e. the wire **deserialized and reached simulation** (a malformed wire errors "failed to deserialize"). So the serialization is byte-correct against the real Solana runtime. The operator utility `broadcast-tool` runs the full pipeline end-to-end (load key -> parse card -> sign -> submit). **PROVEN.**
- **Money moves only by the human's hand.** Every value-moving step emits a *card* and never sends; only the operator, running `broadcast-tool` with a funded key, fires it. This is a deliberate safety boundary, not a missing feature.

What "traction" honestly means today

- **Working software, daily use by the founder.** The harness is real and running.
- **Zero external users / not publicly released.** By choice (energy containment + on-chain hardening), not for lack of a build. This is pre-launch.
- **Reproducible technical proofs** (the router, the commons loop, the chain integration) are in-tree and gated.
- **Capability proofs** (Overwatch one-shot, ARC-AGI score) are founder- demonstrated and are the right things to ask for a recorded, reproducible demo of. The honest ask for diligence: a Loom of the harness one-shotting a fresh task, and the ARC-AGI run with its numbers.